# Multi-ROI Crop Navigation

## Technical Design and Integration Reference

MRN-TD-001 | Revision 1.0

| Document control | Value |
|---|---|
| Document ID | MRN-TD-001 |
| Revision | 1.0 |
| Issue date | 3 November 2026 |
| Software version | 0.1.0 |
| Implementation baseline | 2283540 |
| Project | Multi-ROI Crop Navigation |
| Scope | Detection, navigation analysis, Python integration, and ROS 2 simulation |

System architecture, processing algorithms, configuration, interfaces, execution procedures, and verification records.

# Contents

1. Purpose and system scope 3

2. Architecture and processing sequence 3

3. Image processing and vegetation segmentation 4

4. Adaptive strip search and corridor estimation 5

5. Line and curve representation 5

6. Lookahead corridor memory 6

7. Temporal navigation filtering 7

8. Visual servoing and command generation 7

9. Software interfaces and data contracts 8

10. Configuration reference 9

11. Installation and execution 10

12. ROS 2 and Gazebo integration 11

13. Verification and traceability 12

14. Software delivery and maintenance 13

15. References and supporting documents 13

# 1. Purpose and system scope

Multi-ROI Crop Navigation estimates the travelling corridor between crop rows from camera images. It combines vegetation segmentation, adaptive regions of interest, geometric fitting, temporal processing, and visual servoing. Its outputs support visual inspection, recorded command analysis, Python integration, and camera-based rover control in the included simulator.

This document describes the implementation, its interfaces, and the procedures for installation, execution, and verification. It is intended for software engineers, robotics integrators, and technical reviewers.

The repository supplies three execution paths:

| Execution path | Input | Principal output |
|---|---|---|
| Image detection | Image directory or glob | Segmentation composites and detection CSV |
| Navigation analysis | Image sequence, video, or camera | Linear/angular commands, overlays, and telemetry |
| ROS 2 simulation | Gazebo camera and odometry | Rover commands, live overlays, and simulation logs |

`multi_roi.py` preserves the reference reimplementation of the published adaptive MultiROI method. `test_multi_roi.py` contains the extended detector. The remaining runtime modules provide memory, filtering, control, and runners. Original module names are retained to preserve imports and simulator paths.

# 2. Architecture and processing sequence

The detector and navigation layers are separate. A detector result can be visualized directly or supplied to the navigation pipeline.

```
BGR image
    |
    v
Border crop -> ExG -> Otsu -> morphology -> component filtering
    |
    v
Bottom-to-top strip search -> flanking rows -> corridor geometry
    |
    v
Line / spline fit -> visual features
    |
    v
Optional lookahead memory -> optional temporal filter
    |
    v
Visual servo controller -> command smoothing -> outputs
```

For a continuous stream with lookahead enabled, predicted strip bins are also provided to the detector before its strip search. This forms a feedback path from corridor memory to the next frame's region selection.

The per-frame sequence in `process_image()` is:

1. Request the optional lookahead prediction.
2. Run detection and retain the raw result dictionary.
3. Convert navigation geometry to image features.
4. Compute optional curvature feedforward.
5. Update the optional map and prepare its corrected measurements.
6. Update the optional temporal state.

7. Calculate the linear and angular command.
8. Assemble diagnostics and, when requested, draw the overlay.

| Module | Responsibility |
|---|---|
| `test_multi_roi.py` | Segmentation, strip search, geometry, composites, detection CLI |
| `multi_roi.py` | Reference detector and its image CLI |
| `lookahead_corridor_map.py` | Per-strip corridor memory and prediction |
| `temporal_filter.py` | State filtering and a standalone command smoother |
| `mr_vs.py` | Feature conversion and visual servo control |
| `run_mr_navigation.py` | Shared frame processor and offline runners |
| `mr_navigation_stack.py` | Stateful stack and kinematic pose integration |
| `tests/sim_ros2` | Camera, control, worlds, and ROS 2 launch integration |

The kinematic helper integrates commands into a pose. Gazebo supplies the rendered-camera feedback used for closed-loop simulation.

# 3. Image processing and vegetation segmentation

## 3.1 Input and coordinate preparation

The detector accepts an OpenCV BGR image, normally an unsigned 8-bit array with shape `(height, width, 3)`. The default border crop removes 2% from each edge. The result records the crop offset as `(dx, dy)`.

Detector geometry is expressed in cropped-image coordinates. The feature conversion routine adds the crop offset when constructing full-image points. Image x increases to the right and image y increases downward. Pixel distances remain image-space measurements throughout the detector.

## 3.2 Excess-green index

The default raw index is calculated from floating-point BGR channels:

```
ExG = 2*G - R - B
ExG8 = clip(ExG, 0, 255)
```

An optional normalized index divides each channel by `R + G + B`, replacing a zero denominator with one. The normalized formulation is:

```
ExG_normalized = 1.8*g - 1.1*r - 0.8*b
ExG8 = 255 * clip(ExG_normalized, 0, 1.8) / 1.8
```

Otsu thresholding converts the index to a binary mask with foreground value

255. The opening operation uses the implementation's 4 by 4 anti-diagonal kernel.

## 3.3 Morphology modes

| Mode | Operations |
|---|---|
| `paper` | Anti-diagonal morphological opening |
| `struct` | Opening followed by strip-band component filtering |
| `robust` | Oriented opening and component-attribute filtering |

In `struct` mode, column projections establish candidate row bands. Connected components are assessed as whole components against bands from their own and adjacent strips. Components intersecting row structure are

retained. The implementation also retains components below the component-assessment area floor, large canopy components, and components for which no supporting strip bands are available.

Anchor-based post-scrubbing is a separate, opt-in image-CLI stage. Its scorers include Isolation Forest and gated MAD-z. It is not part of the default single-pass detector configuration.

# 4. Adaptive strip search and corridor estimation

## 4.1 Strip geometry

The search proceeds from the bottom of the cropped mask upward through `N` horizontal strips. With cropped height `H` and coverage fraction `c`, strip height is derived from `H*c/N`, with a minimum of four pixels. The default configuration uses ten strips and covers the bottom 75% of the image.

The first search window spans the image width unless `init_window_frac` restricts it. Subsequent windows follow the selected flanking bands. The current midpoint starts at the horizontal image centre.

## 4.2 Projection and clustering

Within a search window, the column projection counts foreground pixels:

```
Z(x) = number of foreground pixels in column x
T = mean(Z) + std(Z)
Z(x) = 0 when Z(x) < T
L = max(2, int(l_frac * cropped_width))
```

If the threshold would remove every column in a nonzero profile, the threshold falls back to the profile's 90th percentile. Surviving columns separated by a gap smaller than `L` are clustered into bands.

A flank-mass gate removes clusters whose surviving projection mass is below the configured fraction of the strip mass. The default fraction is 0.01. The nearest cluster on each side of the current midpoint forms the flank pair.

## 4.3 Region renewal

For a two-sided selection, the flank span `D` is the distance from the left cluster's outer left edge to the right cluster's outer right edge. The next window expands by 10% of `D` on either side, and its centre is renewed from the mean of the two cluster centres.

A merged-flank operation can trim an unusually wide cluster when a sustained interior valley separates it from adjacent vegetation. With one-sided or missing observations, the previous search window and midpoint are reused. A lookahead prior can suppress a renewed midpoint when its position or span exceeds the configured gates.

## 4.4 Fit evidence

Normal navigation evidence uses two-sided selections with flank span no greater than 1.6 times the measured median span. The first strip is excluded by default, and `ignore_initial` can exclude additional bottom strips. The detector retains accepted and rejected point sets for inspection.

When fewer than two points qualify, the implementation fits fallback strip midpoints. Consequently, `q_accepted` describes membership in the fitted set; `strip_profile.two_sided` and related evidence fields describe the observations that produced it.

# 5. Line and curve representation

## 5.1 Total least squares line

The extended detector fits its line using total least squares. It centres the point set, applies singular value decomposition, and selects the principal axis. Perpendicular residuals are used for iterative rejection, with up to three removals and a threshold based on 2.5 residual standard deviations.

The returned representation is:

```
nav_line = (slope, intercept)
y = slope*x + intercept
```

The implementation uses a steep finite-slope representation for a vertical principal axis. The reference detector uses its original line-fitting path. Flanking detection lines are represented separately from the navigation path.

### 5.2 Cubic corridor curve

When at least four accepted points are available and curve fitting is enabled, the detector constructs a cubic regression spline using SciPy's `LSQUnivariateSpline`. It samples the curve at four-pixel intervals in y and extends it beyond the measured range using bounded endpoint slopes.

The curve is blended toward the TLS reference where its maximum deviation exceeds an envelope derived from 15% of the measured median flank span. The result is returned as sampled `(x, y)` pairs. A line remains available as the reference representation.

Feature conversion uses the lowest sampled segment to calculate the curve heading. The detector's reported base angle is a separate diagnostic evaluated at the deepest measured spline position.

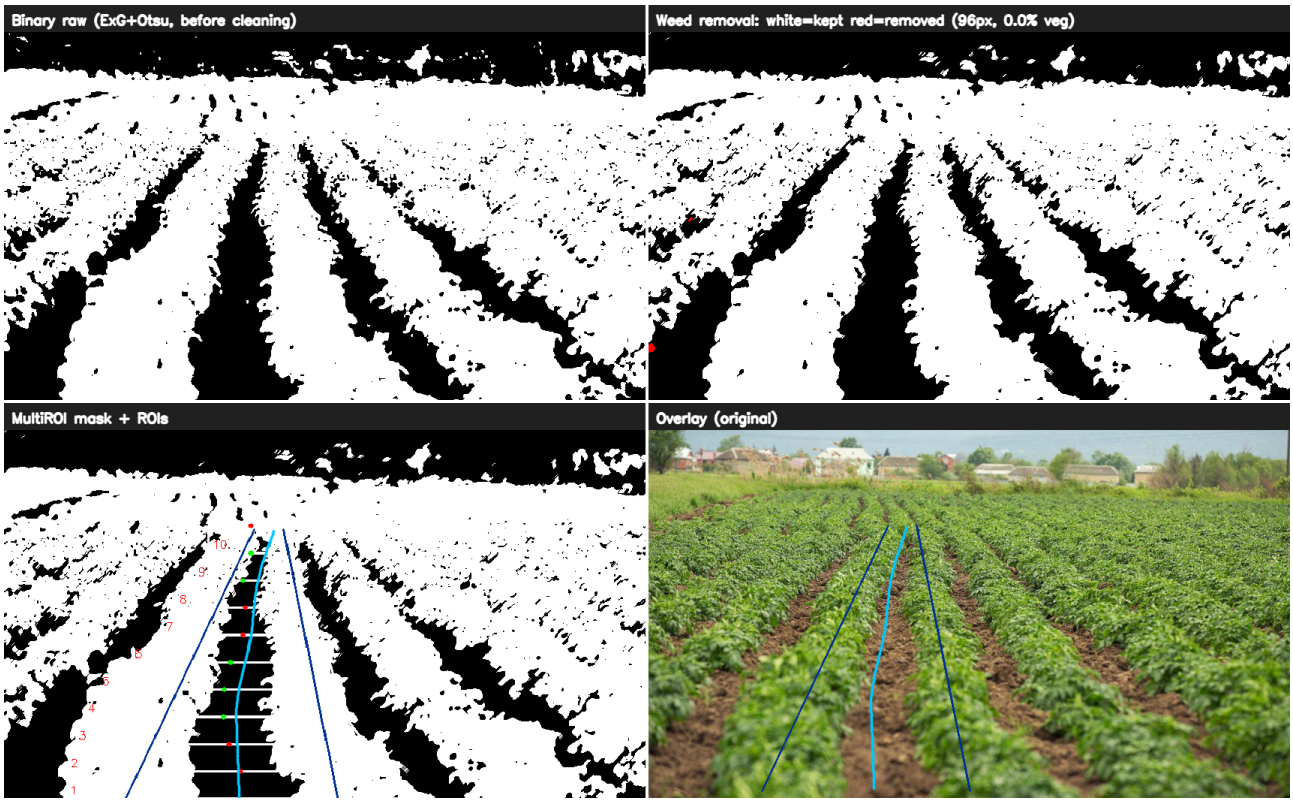



*Figure 1. Segmentation, component removal, strip geometry, and navigation fitting on the bundled field photograph.*

## 6. Lookahead corridor memory

`LookaheadCorridorMap` maintains one bin per configured strip. A bin contains centre, width, flank positions, confidence, age, and evidence state.

Prediction shifts the previous map by a configured number of image pixels per frame. The default shift is 14 pixels. Source bins are selected or interpolated using the shift expressed in strip units. Optional profile matching can choose an alternative shift. The shift is an image-space parameter rather than a physical odometry measurement.

The update compares observed centre and width against the prediction. It also counts upper-strip support for the previous and candidate bottom geometry. The decision is reported through `map_status`.

| Status | Processing interpretation |
|---|---|
| `no_map` | Initialize from observations without a sufficiently confident prior |
| `map_accept` | Accept geometry consistent with the prediction |
| `map_hold` | Retain predicted bottom geometry supported by upper strips |
| `occlusion_hold` | Retain predicted geometry during missing bottom observations |
| `map_pending` | Accumulate support for conflicting geometry |
| `map_switch` | Blend toward a candidate with sufficient accumulated support |
| `held_no_line` | Report the low-confidence bottom-observation fallback |

The map returns corrected geometry, predictions, bin states, innovations, support counts, and confidence. The runner passes corrected values to the optional temporal filter while retaining raw detector diagnostics.

# 7. Temporal navigation filtering

`TemporalNavigationFilter` maintains bottom x, heading, corridor width, lateral feature, confidence, and pending-candidate state. It filters these quantities rather than the potentially steep line slope and intercept.

The default prediction holds the previous state. Optional motion-gain terms can incorporate the supplied angular command and frame interval. Both motion gains default to zero.

Innovation thresholds are evaluated in pixels, radians, and corridor-width units:

```
position_gate = max(18 px, 0.10*image_width, 0.45*max(10 px, previous_width))
heading_gate = 12 degrees
width_gate = 0.30*max(10 px, previous_width)
```

Large position or heading innovations enter the pending-candidate path. Candidate coherence, a count threshold of `persist_frames + spike_confirm_frames`, and the current two-sided evidence count govern acceptance. The default count threshold is seven, and the evidence floor is seven two-sided strips.

Ordinary accepted measurements use exponential moving averages. Width-only anomalies and low-evidence measurements with small position/heading innovations use a slower update path. Missing-line input retains the predicted geometry and decays confidence. The output reports a decision such as `accepted`, `pending`, `pending_accepted`, or `held_no_line`.

The filter exposes `reset()` for stream discontinuities. Stateful objects belong to one continuous camera stream and are reinitialized when the sequence is restarted.

# 8. Visual servoing and command generation

## 8.1 Image features

`MultiROIVS.nav_line_to_feature()` converts fitted geometry into points `P` and `Q` and the feature vector `F`:

```
P = lower reference point in full-image coordinates
Q = upper reference point or adjacent curve sample
X = P.x - image_width/2
Y = P.y - image_height/2
Theta = wrap(pi/2 - atan2(P.y-Q.y, Q.x-P.x))
F = [X, Y, Theta]
```

X is the lateral image feature in pixels. `Theta` is the image heading in radians. The controller uses X and heading; Y is retained in the interface. The positive angular-command convention follows the ROS planar yaw convention.

### 8.2 Control law

Let `e = X / image_width`, h be the configured lateral heading-gate width, `I` the lateral integral, and `ff` optional curvature feedforward:

```
gate = 1 / (1 + (e/h)^2)          when h > 0; otherwise gate = 1
w_raw = -(lambda_x*e + lambda_theta*Theta*gate) - ki*I + ff
```

The integral accumulates normalized lateral error when confidence meets its threshold. It is clamped and otherwise decays. The raw angular command is clamped before optional smoothing.

### 8.3 Command processing

With smoothing enabled, angular processing applies a low-pass update, a per-frame change bound based on `max_w_rate*dt`, angular clamping, and a deadband. Forward velocity is scaled by confidence when that option is enabled:

```
v = vf_des * (v_min_scale + (1-v_min_scale)*confidence)
```

The command pair uses metres per second for `v` and radians per second for `w`. Curvature feedforward is available through `process_image()` and the simulator adapter. It is disabled by default in the offline frame-processing function.

## 9. Software interfaces and data contracts

### 9.1 Detector interface

```
from test_multi_roi import MultiROIDetector

detector = MultiROIDetector(morph="struct", vertical_coverage=0.75)
result = detector.detect(bgr_image)
```

| Result field | Contract |
|---|---|
| `binary` | Cropped foreground mask |
| `crop_offset` | `(dx, dy)` offset to full-image coordinates |
| `nav_line` | TLS line tuple or `None` |
| `nav_curve` | Sampled spline coordinates or `None` |
| `q_accepted`, `q_rejected` | Fit-membership point sets |
| `median_width`, `bottom_width` | Pixel flank spans |
| `n_two_sided` | Measured strip count after the detector's selection rules |
| `strip_profile` | Per-strip geometry, evidence, suppression, and region fields |
| `time_ms` | Detector execution time in milliseconds |

### 9.2 Frame-processing interface

`process_image()` accepts the BGR frame, detector, controller, and optional filter/map instances. It also accepts `dt`, `last_w`, and feedforward settings. It returns the detector result, raw and selected feature vectors, reference points, commands, diagnostics, overlay, and optional filter/map outputs.

```
from pathlib import Path
import cv2
from test_multi_roi import MultiROIDetector
from mr_vs import MultiROIVS, MRVSParams
from temporal_filter import TemporalNavigationFilter, TemporalFilterParams
from run_mr_navigation import process_image

source = Path("examples/field/photo_7_2026-08-18_17-17-57.png")
image = cv2.imread(str(source))
```

```
if image is None:
    raise FileNotFoundError(source)
height, width = image.shape[:2]

detector = MultiROIDetector(morph="struct")
controller = MultiROIVS(MRVSParams(width=width, height=height))
state_filter = TemporalNavigationFilter(
    TemporalFilterParams(image_width=width, image_height=height)
)
output = process_image(
    image, detector, controller, t_filter=state_filter,
    dt=0.05, last_w=0.0, draw=True,
)
Path("outputs/integration").mkdir(parents=True, exist_ok=True)
if not cv2.imwrite("outputs/integration/overlay.png", output["overlay"]):
    raise RuntimeError("Could not write overlay")
print(output["v"], output["w"], output["info"]["status"])
```

For subsequent frames, reuse the objects and supply the previous command as `last_w`. Set `dt` from the source timing in integrations that provide it. The file-video runner uses elapsed processing time for its control interval. The simulator adapter leaves `dt` unspecified in `process_image()`, which uses the controller/filter timing paths. These timing choices should be recorded with each configuration.

### 9.3 Output contracts

Detection produces one four-panel composite per readable image and a detection CSV. Folder navigation produces navigation, composite, and combined images, plus `nav_commands.csv`. File-video mode produces a processed video and CSV. Camera mode records CSV diagnostics without recording an output video.

CSV schemas are specific to each runner. The header is the schema reference. Angles in CSV diagnostics are generally degrees; internal heading features are radians. Corridor widths and lateral image errors are pixels. Simulator cross-track error is expressed in metres relative to the configured lane.

## 10. Configuration reference

### 10.1 Execution-path defaults

| Parameter | Image detector CLI | Navigation CLI | Python library | Simulator adapter |
|---|---|---|---|---|
| Morphology | `struct` | `struct` | `paper` | `paper` |
| Strips | 10 | 10 | 10 | 10 |
| Vertical coverage | 0.75 | 0.75 | 0.75 | Preset/env configured |
| Spline | Enabled | Enabled | Enabled | Enabled unless line mode |
| Temporal filter | Not used | Video on; folder off | Explicit construction | Enabled |
| Lookahead map | Not used | Video on; folder off | Explicit construction | Disabled |
| Lateral control gain | Not used | 10.0 | 2.0 in `MRVSParams` | 2.0 default |
| Angular command bound | Not used | 0.50 rad/s | 0.60 rad/s | 0.60 rad/s |
| Reference speed | Not used | 0.20 m/s | 0.20 m/s | 0.20 m/s |

Library settings in this table refer to constructor/dataclass defaults. Simulator world sidecars, launch arguments, and environment variables can supply different settings. Record the effective configuration with the run.

### 10.2 Memory and filtering parameters

| Parameter | Default | Unit or meaning |
|---|---|---|
| `LookaheadParams.shift_px` | 14.0 | Image pixels per processed frame |
| `max_center_gate_frac` | 0.35 | Fraction of reference corridor width |
| `max_width_gate_frac` | 0.35 | Fraction of reference corridor width |
| `accept_frames` | 4 | Map candidate persistence count |
| `accept_bins` | 3 | Supporting upper bins |
| `conf_decay` | 0.97 | Prediction confidence factor |
| `persist_frames` | 4 | Temporal candidate count component |
| `spike_confirm_frames` | 3 | Additional temporal confirmation count |
| `commit_min_two_sided` | 7 | Evidence floor for large-jump acceptance |
| `alpha_x`, `alpha_theta` | 0.35 | Accepted-state smoothing factors |
| `alpha_width` | 0.30 | Width smoothing factor |
| `w_alpha` | 0.35 | Angular-command low-pass factor |
| `max_w_rate` | 1.2 | Radians per second squared |
| `w_deadband` | 0.02 | Radians per second |
| `v_min_scale` | 0.45 | Forward-speed scale at zero confidence |

## 11. Installation and execution

The offline package targets Python 3.10 through 3.12. Its runtime dependencies are NumPy, OpenCV, SciPy, and scikit-learn. Version ranges are declared in `pyproject.toml`; the reference direct dependency versions are recorded in `constraints-tested.txt`.

### 11.1 Offline installation

```
git clone https://github.com/A-C-Simon/multi-roi-crop-navigation.git
cd multi-roi-crop-navigation
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install .
```

Ubuntu environments may require the OpenCV runtime libraries described in the installation guide. Use a separate environment for ROS.

### 11.2 Installed commands

| Command | Entry point |
|---|---|
| `multiroi-detect` | `test_multi_roi:main` |
| `multiroi-nav` | `run_mr_navigation:main` |
| `multiroi-reference` | `multi_roi:main` |

```
multiroi-detect --input examples/field --results_dir outputs/field-example
multiroi-nav --input examples/field --output outputs/navigation
multiroi-reference --input examples/field --results_dir outputs/reference
```

For file video, supply input geometry appropriate to the source:

```
multiroi-nav --input /path/to/video.mp4 --video --width 640 --height 480 --output outputs/video
```

Use `--help` for the full interface. Always supply an input path. The preserved historical default input points to `../../Photos`.

### 11.3 File selection and outputs

Directory detection selects lowercase `.png` and `.jpg` names. Folder navigation also selects `.jpeg`. An explicit quoted glob supports other OpenCV-readable names and formats. The detection CLI downsizes its longest image side to at most 1920 pixels unless configured otherwise.

The image detector retains a name-based rotation rule for `bev5`, `bev6`, and `bev7`. Use `--rotate_names ''` when applying it to an unrelated naming scheme. Use separate output directories for independently retained runs.

## 12. ROS 2 and Gazebo integration

The included simulation workspace targets ROS 2 Humble and Gazebo Classic 11 on Ubuntu 22.04. The full environment procedure is documented in simulation.md.

The simulated camera feeds the same detector and controller modules through `mrsim/pipeline.py`. The adapter constructs its own configuration and publishes commands through the navigation node.

| ROS topic | Message type | Function |
|---|---|---|
| `/camera/image_raw` | `sensor_msgs/Image` | Primary camera input |
| `/camera_back/image_raw` | `sensor_msgs/Image` | Optional rear camera input |
| `/odom` | `nav_msgs/Odometry` | Rover pose and velocity |
| `/cmd_vel` | `geometry_msgs/Twist` | Rover command |
| `/multiroi/overlay` | `sensor_msgs/Image` | Primary diagnostic view |
| `/multiroi/overlay_rear` | `sensor_msgs/Image` | Rear diagnostic view |
| `/reset_rover` | `std_msgs/Empty` | Rover reset request |

When the ToF guard is enabled, upstream control uses `/cmd_vel_raw` and the guard publishes `/cmd_vel`. Range channels include side-facing and angled front sensors. The guard computes a clearance-based yaw override and a forward speed cap.

The world presets include straight, curved, circular, and zigzag fields. Sidecar JSON files hold spawn and lane defaults. Plant models are included at the paths expected by the launch files. The simulator also includes scripted row transitions, rear-camera processing, teleoperation, and reset handling.

From a configured simulation shell at the repository root:

```
export MULTIROI_DIR="$PWD"
cd tests/sim_ros2
./run_sim.sh --probe --out /tmp/multiroi-probe
```

After inspecting the camera output, run a bounded lane-following session:

```
./run_sim.sh --straight 5 --seconds 30 --out /tmp/multiroi-run
```

The simulation produces `nav_run.csv` and periodic overlay images. The CSV includes odometry, lane cross-track error, visual features, confidence, control commands, and detector evidence. The ToF option adds its own telemetry.

# 13. Verification and traceability

## 13.1 Regression suite

The repository retains eight synthetic checks:

| Check | Exercise |
|---|---|
| Missing plants | Transient row-evidence reduction |
| Persistent lane shift | Filter response to sustained displacement |
| Width expansion | Sudden corridor-width change |
| Straight corridor | Stable command on a centred corridor |
| Slanted corridor | Nonzero steering on a slanted scene |
| Vertical coverage | Drawing clipped to the requested coverage |
| Heading spike | Brief heading excursion handling |
| Short pulse | Short square-pulse handling |

```
python -m pip install -e '.[dev]'
python -m pytest
python scripts/verify_snapshot.py
```

## 13.2 Integration checks

The checked example workflow generates two image inputs and a 40-frame video. It executes the installed commands and verifies readable image composites, CSV record counts, and the encoded output video frame count.

```
python examples/generate_sample.py
multiroi-detect --input examples/data --results_dir outputs/detection
multiroi-nav --input examples/data/corridor.mp4 --video --output outputs/video
python scripts/check_example_outputs.py
```

## 13.3 Recorded delivery evidence

| Item | Recorded result |
|---|---|
| Existing synthetic suite | Eight checks passed |
| Python CI matrix | 3.10, 3.11, and 3.12 passed |
| Preserved-source verification | 45 source and asset checksums matched |
| Example image outputs | Two readable composites |
| Example video output | 40 readable frames and matching telemetry |
| Distribution builds | Wheel and source archive generated |
| Field example | Bundled `photo_7` and reproducible detector composite |
| Demonstration | 62.52-second simulation recording and animated extract |

These entries identify the verification performed for the repository delivery. The validation page, CI workflow, scripts, and source snapshot provide the corresponding procedures and records.

### 13.4 Run records

Retain the input identifier, software revision, dependency versions, effective configuration, image dimensions, timing source, output files, and matching CSV rows. For simulation, also retain the world preset, spawn sidecar, launch arguments, and odometry reference. Historical CSV angles describe estimated geometry; use their field definitions when comparing runs.

## 14. Software delivery and maintenance

The repository is installable through `pyproject.toml`. Build the distributable artifacts with:

```
python -m pip install -e '.[dev]'
python -m build
```

The wheel supplies the runtime modules and command entry points. The source archive additionally carries documentation, examples, simulator files, plant models, and historical tables selected by `MANIFEST.in`.

The import record in `source-snapshot.json` identifies the original repository, revision, directory, and SHA-256 checksums. Packaging and documentation have separate revisions from the preserved runtime implementation. Changes to runtime behavior should be tracked with a corresponding source revision, configuration update, and regression evidence.

To rebuild the PDF edition of this document:

```
python -m pip install '.[docs]'
python scripts/build_technical_documentation.py
```

The build uses the Markdown document as its source and writes `docs/technical-documentation.pdf`. It does not execute the detector or change runtime files.

## 15. References and supporting documents

1. Zhou Y, Yang Y, Zhang B L, Wen X, Yue X, Chen L Q. Autonomous detection of crop rows based on adaptive multi-ROI in maize fields. International Journal of Agricultural and Biological Engineering, 2021, 14(4), 217-225. DOI.

2. Original project and source history.

3. Installation, usage, and Python interface.

4. Architecture, output definitions, and simulation setup.

5. Validation procedures and source snapshot.

6. Research experiment archive and project provenance.

7. Third-party notices.